Практическая работа № 2.3 Модель на PHP.

Цель: Оптимизация разработки при помощи архитектурных паттернов разработки.

Теория:

Контроллер (Controller)

Он обеспечивает взаимодействие с системой: обрабатывает действия пользователя, проверяет полученную информацию и передает ее модели. Контроллер определяет, как приложение будет реагировать на действия пользователя. Также контроллер может отвечать за фильтрацию данных и авторизацию.

Практика:

Задание 1.

Создаём БД

sql

Copy

Download

```sql
CREATE DATABASE mvc_demo CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE mvc_demo;

CREATE TABLE articles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO articles (title, content) VALUES
('Первая статья', 'Это содержимое первой статьи про MVC.'),
('Вторая статья', 'Здесь мы разбираем контроллеры.'),
('Третья статья', 'А тут — про представления.');
```

config/db.php

php

Copy

Download

```php
<?php
return [
    'host'     => 'localhost',
    'dbname'   => 'mvc_demo',
    'user'     => 'root',
    'password' => '',
    'charset'  => 'utf8mb4',
];
```

app/Core/Database.php (одно подключение на всё приложение)

php

Copy

Download

```php
<?php

namespace App\Core;

use PDO;

class Database
{
    private static ?PDO $pdo = null;

    public static function pdo(): PDO
    {
        if (self::$pdo === null) {
            $config = require __DIR__ . '/../../config/db.php';
            $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};c
harset={$config['charset']}";

            self::$pdo = new PDO($dsn, $config['user'], $config['password'],
[
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            ]);
        }
        return self::$pdo;
    }
}
```

app/Models/Article.php

php

Copy

Download

```php
<?php
```

```php
namespace App\Models;

use App\Core\Database;

class Article
{
    public function all(): array
    {
        return Database::pdo()
            ->query("SELECT * FROM articles ORDER BY created_at DESC")
            ->fetchAll();
    }

    public function find(int $id): ?array
    {
        $stmt = Database::pdo()->prepare("SELECT * FROM articles WHERE id = ?
");
        $stmt->execute([$id]);
        return $stmt->fetch() ?: null;
    }
}
```

Контроллер финальный

php

Copy

Download

```php
<?php

namespace App\Controllers;

use App\Core\Controller;
use App\Models\Article;

class ArticleController extends Controller
{
    private Article $model;

    public function __construct()
    {
        $this->model = new Article();
    }

    public function index(): void
    {
        $this->view('articles/index', [
            'articles' => $this->model->all(),
        ]);
    }

    public function show(int $id): void
    {
        $article = $this->model->find($id);

        if (!$article) {
            http_response_code(404);
            exit('Статья не найдена');
        }
```

```
        $this->view('articles/show', ['article' => $article]);
    }
}
```

Теоретические вопросы:

1. Контроллер **не изменил форму вызова view** — он всё так же отдаёт массив. Меняется только **источник данных**.
2. Модель — единственное место, где живёт SQL.
3. Если завтра статьи будут в JSON-файле или в API — меняется только модель, контроллер и view не трогаются.

Задание 2.

Создаём БД

```
CREATE DATABASE mvc_demo CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE mvc_demo;

CREATE TABLE articles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO articles (title, content) VALUES
('Первая статья', 'Это содержимое первой статьи про MVC.'),
('Вторая статья', 'Здесь мы разбираем контроллеры.'),
('Третья статья', 'А тут — про представления.');
```

config/db.php

```
<?php
return [
    'host'     => 'localhost',
    'dbname'   => 'mvc_demo',
    'user'     => 'root',
    'password' => '',
    'charset'  => 'utf8mb4',
];
```

app/Core/Database.php (одно подключение на всё приложение)

```
<?php

namespace App\Core;

use PDO;

class Database
{
```

```
    private static ?PDO $pdo = null;

    public static function pdo(): PDO
    {
        if (self::$pdo === null) {
            $config = require __DIR__ . '/../../config/db.php';
            $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};c
harset={$config['charset']}";

            self::$pdo = new PDO($dsn, $config['user'], $config['password'],
[
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            ]);
        }
        return self::$pdo;
    }
}
```

app/Models/Article.php

```
<?php

namespace App\Models;

use App\Core\Database;

class Article
{
    public function all(): array
    {
        return Database::pdo()
            ->query("SELECT * FROM articles ORDER BY created_at DESC")
            ->fetchAll();
    }

    public function find(int $id): ?array
    {
        $stmt = Database::pdo()->prepare("SELECT * FROM articles WHERE id = ?
");
        $stmt->execute([$id]);
        return $stmt->fetch() ?: null;
    }
}
```

Контроллер финальный

```
<?php

namespace App\Controllers;

use App\Core\Controller;
use App\Models\Article;

class ArticleController extends Controller
{
    private Article $model;

    public function __construct()
    {
        $this->model = new Article();
```

```
    }

    public function index(): void
    {
        $this->view('articles/index', [
            'articles' => $this->model->all(),
        ]);
    }

    public function show(int $id): void
    {
        $article = $this->model->find($id);

        if (!$article) {
            http_response_code(404);
            exit('Статья не найдена');
        }

        $this->view('articles/show', ['article' => $article]);
    }
}
```

Теоретические вопросы:

1. Контроллер **не изменил форму вызова view** — он всё так же отдаёт массив. Меняется только **источник данных**.
2. Модель — единственное место, где живёт SQL.
3. Если завтра статьи будут в JSON-файле или в API — меняется только модель, контроллер и view не трогаются.

ЛИТЕРАТУРА

1. https://blog.skillfactory.ru/glossary/mvc/ - Что такое MVC.
2. https://www.php.net/manual/ru/ - Официальная русская документация по PHP.
3. https://metanit.com/php/tutorial- Учебник по PHP.